Reasoning and Discovery — Derivation, Abduction and Runtime Acquisition

Irfan Mahir — Independent Researcher, Dhaka, Bangladesh — irfan@furylogic.com

Contents

Reasoning and Discovery — Derivation, Abduction and Runtime Acquisition **1**
Contents of This Volume 1
RADE: The Recursive Axiom-Derivation Engine — Deduction over Prediction in a Persistent Truth Graph 2
Morphological Abduction Architecture (MAA): Overcoming Topological Local Minima in Graph-Rewriting Cognition Engines 6
The L9-Cognitive Graph: Nine Layers from Lexical Decomposition to Paradigm Generation 9
LTO: Live Training Over Manifolds — A Non-Transformers Continuous Learning Architecture 14

Reasoning and Discovery — Derivation, Abduction and Runtime Acquisition

Irfan Mahir
Independent Researcher, Dhaka, Bangladesh
irfan@furylogic.com

Proof-passes over a truth graph, handling the unknown, and acquiring knowledge while running.

Contents of This Volume

#	Paper	File
1	RADE: The Recursive Axiom-Derivation Engine — Deduction over Prediction in a Persistent Truth Graph	`RADE.md`
2	Morphological Abduction Architecture (MAA): Overcoming Topological Local Minima in Graph-Rewriting Cognition Engines	`maa.md`

#	Paper	File
3	The L9-Cognitive Graph: Nine Layers from Lexical Decomposition to Paradigm Generation	`l9.md`
4	LTO: Live Training Over Manifolds — A Non-Transformers Continuous Learning Architecture	`lto.md`

RADE: The Recursive Axiom-Derivation Engine — Deduction over Prediction in a Persistent Truth Graph

> *"Novelty by guessing and novelty by formal expansion look alike from the outside. Only one of them can show its work."*

Abstract

A Transformer-style attention mechanism remains a statistical prediction machine regardless of how its routing is rearranged: what is computed is a distribution over continuations conditioned on co-occurrence, and changing the plumbing does not change that. This paper specifies **RADE**, an architecture organised around **logical synthesis** rather than probabilistic approximation.

RADE replaces three components at once. Knowledge moves from floating-point weights into an **Axiom Substrate** — a persistent graph of verified relations. Inference moves from a forward pass to a **proof pass** over that graph. And termination moves from a fixed depth to **homeostatic inhibition**, a resource budget whose exhaustion triggers a lateral switch to a different region of the graph rather than an unbounded search.

The consequence that matters is a change of failure mode. A predictive system fails by producing fluent falsehood, which is undetectable from the output alone. RADE fails by **logical inconsistency**, which is detectable, localisable, and attributable to specific premises.

---

1. The Problem with Rearranged Plumbing

Architectural variation within the predictive family — sparse attention, linear attention, state-space substitutions, mixture routing — changes the cost of computing $P(w_n \mid w_1 \dots w_{n-1})$ without changing what that quantity is. A count of how often one symbol followed another is not an account of *why* it did, and no rearrangement of the machinery that computes the count converts it into one.

This is not an argument that prediction is useless. It is an argument about what a system can be asked for. A predictive system can be asked for a continuation. It cannot be asked for a *derivation*, because it does not have one to give — the output was not produced by steps that could be exhibited.

RADE is specified to answer the second kind of request.

---

2. The Axiom Substrate

2.1 Knowledge as structure, not as parameters In a Transformer, knowledge is distributed across floating-point weights and cannot be inspected, edited, or deleted at the granularity of a fact. In RADE it is held explicitly:

- **Formal objects** O — concepts, as graph vertices.
- **Operators** Φ — typed relations, as directed edges.

The distinction from embeddings is persistence semantics. An embedding degrades under continued training: the representation of a concept drifts as unrelated updates accumulate. A graph edge does not. If the substrate holds `A is_a B`, it holds it until something explicitly removes it, and the removal is an event that can be logged.

2.2 Why this matters for correction A predictive model cannot be told that one specific thing it believes is wrong. The belief is not localised, so correction is retraining, and retraining perturbs everything else.

In a truth graph, a wrong fact is an edge. Removing it is $O(1)$, and the consequences of removing it are computable: every derived edge citing it as a premise becomes unsupported and can be retracted by following the provenance chain. Correction is therefore *bounded* rather than global.

---

3. The Proof Pass

The forward pass is replaced by a traversal.

Input. A query — for example, *why is X a superconductor?*

Operation. The engine does not estimate an answer. It searches the graph for a path connecting the queried concepts through typed operators. If a path exists, the path *is* the answer, and every edge along it is either an asserted premise or a derived edge that names the two premises that produced it.

When no path exists. The engine enters a **search-and-constrain** phase: it proposes a candidate link and tests it against the existing axioms. A candidate that contradicts a held axiom is rejected. A candidate that survives is committed as a derived edge, with its provenance recorded.

This is the distinction the paper turns on:

> **Novelty by guessing** produces a plausible string. **Novelty by formal expansion** produces an edge that survived a consistency test and can name what it rests on.

Both look like new output. Only the second can be audited, and only the second can be retracted cleanly when its support is withdrawn.

---

4. Homeostatic Inhibition

An unbounded proof search does not terminate on a large graph. The biological analogue of the fix is inhibitory interneurons, which prevent excitation from propagating without limit.

Resource budget. Every search path is allocated a bounded budget.

Lateral inhibition. A path that yields no result within its budget is abandoned, and the search is displaced to a different region of the graph rather than deepened.

The behavioural effect is the ability to *stop pursuing an unproductive line*, which is a capability rather than a limitation: a system that cannot abandon a path cannot allocate effort. This mechanism is the direct ancestor of the tension-bounded construction budget in the successor architecture, where the budget is derived from a physical observable rather than set as a constant.

---

5. Comparison

Property	Transformer	RADE
Foundation	statistical probabilities	logical axioms
Inference	forward pass	proof pass over a graph

Property	Transformer	RADE
Novelty	interpolation, indistinguishable from hallucination	formal expansion, consistency-tested
Memory	lossy KV cache	persistent graph of hard relations
Correction	retraining, global	edge removal, bounded by provenance
Failure mode	fluent falsehood, **undetectable from output**	logical inconsistency, **detectable and localisable**
Objective	minimise prediction error	maximise logical consistency

The failure-mode row carries the argument. A system whose errors are detectable can be improved by an operator who does not have access to its internals. A system whose errors are fluent cannot.

---

6. Ingestion: The Axiom-ifier

A corpus is not knowledge until it is structured. RADE separates **reading** from **reasoning**: an ingestion stage converts source material into structured triples, and the reasoning engine never touches raw text.

The ingestion stage emits only triples that the substrate can type-check against its operator set. Anything it cannot structure is dropped rather than approximated — a coverage gap is preferable to a fabricated relation, because a gap is visible and a fabrication is not.

The successor architecture implements exactly this split: mechanical AST extraction produces triples from source, and every fact carries a provenance class distinguishing what was asserted from what was derived and what was established by experiment.

---

7. Status and Relation to the Implemented System

RADE is an **architectural proposal**, written before the substrate that now exists. Its components map onto implemented mechanisms as follows:

RADE component	Implemented as	Status
Axiom Substrate	typed concept graph, `data/*.axioms`	**live**
Proof pass	forward chaining with premise citation	**live** — derived edges name their parents
Search-and-constrain	contradiction check before assertion	**live**
Resource budget	tension-bounded construction budget	**live**, now derived from field energy
Lateral inhibition	budget exhaustion ends a derivation chain	**partial** — no region-switching heuristic
Axiom-ifier	mechanical AST extraction	**live**, no ML

The one component not carried across is the explicit region-switching behaviour of §4. The current engine exhausts a budget and stops; it does not redirect to a different subgraph. Whether that redirection is worth implementing is open, and it should not be reported as present.

---

8. Limitations

Ingestion is the bottleneck, not reasoning. The proof pass is cheap; converting unstructured source into typed triples is not. The quality ceiling of the whole system is the quality of that conversion, and this paper does not solve it.

Consistency is not truth. The search-and-constrain phase rejects candidates that contradict held axioms. It cannot reject a candidate that is consistent with a set of axioms that are themselves wrong. Consistency is a necessary condition and nothing more.

Coverage cliff. A query whose concepts are absent from the substrate returns nothing. This is the correct behaviour and it is also a visible limitation; the honest presentation is an explicit gap rather than a degraded guess.

---

9. Conclusion

RADE's claim is narrow and, stated carefully, defensible: an architecture that stores relations rather than parameters, infers by traversal rather than by prediction, and bounds its search by an explicit budget can answer for its outputs in a way a predictive system structurally cannot.

It does not claim to reason better. It claims to reason *auditably* — and to fail in a way that can be seen.

References

- Mahir, I. *From Artificial to Synthetic Intelligence: Principles of Constructive Structural Cognition.* — the tri-layer architecture in which RADE's components are implemented.
- Mahir, I. *JAS: Einstein's Epistemology as a Running Architecture.* — the provenance classes that make retraction bounded.
- Implementation: `src/rade.hpp`.

Morphological Abduction Architecture (MAA): Overcoming Topological Local Minima in Graph-Rewriting Cognition Engines

Author: Irfan Mahir **Date:** July 2026 **Subjects:** Artificial Intelligence (cs.AI); Computation and Language (cs.CL); Logic in Computer Science (cs.LO)

Abstract

Graph-rewriting cognition engines offer a rigid, verifiable alternative to the fluid approximations of statistical large language models. However, when these architectures treat semantic inputs as atomic, unbreakable nodes, they become highly susceptible to topological stagnation. When encountering an undefined concept (an orphan node), standard inference pipelines (deduction, induction, analogy) fail because the search space is mathematically empty, causing the system to infinitely loop its active cache. We propose the Morphological Abduction Architecture (MAA), a framework that breaks the atomic constraint by introducing Sub-Lexical Decomposition and Abductive Synthesis. By dynamically fracturing unknown nodes and mapping their sub-components to existing ontologies, the system is able to hypothesize structural meaning—transforming the engine from a rigid memorizer into a dynamic semantic synthesizer capable of state-dependent cognitive dispatching.

---

1. Introduction

In discrete graph-rewriting systems, cognition is defined by the traversal and construction of edges between known concept nodes. While this ensures strict logical adherence, it introduces a critical vulnerability: the "Atomic Node Trap."

When standard graph engines encounter a novel term—such as a previously unseen acronym or portmanteau—the parser initializes a singular, isolated node $\emptyset_V$. Because this node possesses a degree of zero ($deg(\emptyset_V) = 0$), there are no topological pathways for the inference engine to traverse. Consequently, monolithic evaluation loops (which linearly execute deduce $\rightarrow$ induce $\rightarrow$ analogize pipelines) encounter a void. Lacking a mechanism to process the unknown, the engine falls back onto its highest-activation localized structures, resulting in pathological repetition (e.g., infinitely looping a cached analogy about consciousness regardless of the prompt).

To achieve general reasoning, a system must be capable of synthesizing meaning from the unknown. The Morphological Abduction Architecture (MAA) formalizes the mechanics of "connecting the dots" by physically fragmenting orphan nodes and generating hypothesized graph structures.

2. Component 1: Morphological Decomposition

To process an unknown variable, MAA dictates that a concept node must not be treated as a fundamental particle, but as a composite structure capable of fission.

Let the user introduce an unknown string, C_{unk}. When the parser detects that $deg(C_{unk}) = 0$ within the active ontology O, the engine suspends standard inference and triggers a Decomposition Rule.

The string C_{unk} is fractured into a set of sub-lexical fragments $F = \{f_1, f_2, \dots, f_n\}$ based on linguistic boundaries, syllables, or character clusters. The engine then projects these fragments against the existing ontology to identify partial topological matches:

$$M = \{ f_i \in F \mid \exists n \in O \text{ where } similarity(f_i, n) > \tau \}$$

Case Study (The Superuser Maneuver): If the engine encounters the unknown node `[sudo]`, it evaluates to $deg([sudo]) = 0$. MAA fractures the string into $F = \{su, do\}$. The engine scans the ontology and successfully binds the fragments to known concepts: $su \rightarrow [super_user]$ $do \rightarrow [execute_action]$

The engine has successfully extracted topological anchors from a previously isolated node.

3. Component 2: Abductive Synthesis Graphing

Once morphological anchors are established, the engine cannot use Deduction (which requires absolute axiomatic truth) or Induction (which requires statistical precedence). It must use **Abduction**—the generation of the most likely explanatory hypothesis.

MAA introduces a volatile edge type to the graph schema: `EDGE_HYPOTHESIZED`. Unlike axiomatic edges, hypothesized edges are temporary structural bridges used to test theoretical topologies without corrupting the permanent ontology.

Following the extraction of match set M, the Abductive Synthesis Engine generates a Hypothesis Graph Γ_H.

For the unknown node `[rdma]`, the engine fragments the node into $\{rd, ma\}$. It finds partial matches for $rd \rightarrow [remote/read]$ and $ma \rightarrow [memory/machine]$. The engine synthesizes a cluster of theoretical connections: `[rdma] --EDGE_HYPOTHESIZED--> [remote_memory] [rdma] --EDGE_HYPOTHESIZED--> [read_machine]`

By generating these volatile edges, the node C_{unk} is no longer orphaned. The search space is populated, allowing the cognitive loop to resume execution, score the hypotheses against the surrounding context, and select the highest-probability path.

4. State-Dependent Cognitive Dispatching

The final requirement of MAA is the abolition of the monolithic execution loop. A static `respond()` pipeline that applies the same sequence of rules to every input is fundamentally a memorization behavior. True cognition requires the processing architecture to adapt to the topology of the current graph state.

MAA replaces the static loop with a dynamic Cognitive Dispatcher function, Δ. Before any inference is executed, Δ evaluates the macroscopic geometry of the active graph and routes the execution thread to specialized sub-engines:

1. **State α (Topological Collision):** If the input graph introduces an edge that directly opposes an axiomatic invariant (e.g., `machine --has--> biology`), Δ routes to the **Resolution Pipeline** to isolate and reject the contradiction.
2. **State β (Orphan Detected):** If the input graph contains a node where $deg(n) = 0$ (e.g., `[rdma]`), Δ routes to the **Abductive Pipeline** (Decomposition and Synthesis) to map the unknown.
3. **State γ (Connected Query):** Only when the input graph is fully integrated into the existing ontology does Δ route to the **Standard Inference Pipeline** to deduce new paths.

$$\Delta(\Gamma_{active}) \rightarrow \begin{cases} Pipeline_{Resolution} & \text{if } \exists \text{ contradiction in } \Gamma \\ Pipeline_{Abductive} & \text{if } \exists n \in \Gamma \text{ where } deg(n) = 0 \\ Pipeline_{Standard} & \text{otherwise} \end{cases}$$

5. Conclusion

The Morphological Abduction Architecture demonstrates that the inability of discrete graph engines to handle unknown variables is not a limitation of discrete mathematics, but a failure of parsing granularity and execution routing. By allowing the engine to fracture concepts into their morphological roots and abductively synthesize hypothesized edge relationships, we eliminate topological local minima. Furthermore, implementing a State-Dependent Cognitive Dispatcher ensures that the machine actively "reads" its own structural state before acting, permanently transitioning the architecture from a rigid data-retrieval script into an adaptive, synthesizing cognition engine.

The L9-Cognitive Graph: Nine Layers from Lexical Decomposition to Paradigm Generation

> *"A graph that answers `[Apple] --is_a--> [Fruit]` is a dictionary. Maxwell did not consult a dictionary."*

Abstract

Probabilistic next-symbol prediction confines a system to interpolating within existing knowledge. We propose the **L9-Cognitive Graph**, a neuro-symbolic architecture organised as nine discrete layers of traversal logic over a deterministic symbolic graph, ascending from lexical decomposition to the generation of new axiomatic roots.

The organising claim is that breakthrough reasoning is not semantic retrieval but **cross-domain isomorphism**: recognising that the mathematical structure of one domain describes the unresolved anomalies of another. Achieving this requires the engine to move from *constraint enforcement* to *abductive constraint generation*.

This paper specifies all nine layers — each with its input, operation, output and failure mode — rather than listing them. Section 5 states, layer by layer, which are implemented, which are partial, and which remain proposals. Four are live, two are partial, and three are not built.

---

1. The Limits of Semantic Retrieval

A graph database mapping `[Apple] --is_a--> [Fruit]` is a semantic dictionary: it returns what was stored, structured. Human breakthrough logic does not work this way. Maxwell's unification of electricity and magnetism was not a lookup; it was the recognition that one domain's mathematical structure resolved another's anomalies.

Reproducing that requires two capabilities a dictionary lacks. The engine must be able to notice that its own knowledge is **structurally inconsistent** rather than merely incomplete, and it must be able to **propose structure that was never asserted** in response.

The layers below are ordered by how far each departs from retrieval.

---

2. Biological Motivation: Two Topologies

The architecture borrows two structures rather than a metaphor.

Neocortical columns — the local graph. The brain processes localised, highly deterministic rules: syntax, basic physics. These correspond to densely connected subgraph clusters with near-unit edge weights, where traversal is reliable and cheap.

The Default Mode Network — the abductive engine. The DMN activates during undirected thought and deep problem-solving, searching for low-probability, long-distance connections across disparate columns. It corresponds to an asynchronous background process running bridge-finding across structurally similar but semantically distant subgraphs.

The functional distinction that matters: **the local graph answers, the DMN wanders.** A system with only the first can never surprise itself.

---

3. The Nine Layers

Each layer is specified by what it takes, what it does, what it emits, and how it fails. Layers are cumulative: Ln presupposes L1..n−1.

L1 — Lexical / Morphological Decomposition **Input.** A token that resolves to no node. **Operation.** Fracture the string into sub-lexical fragments — morphemes, syllables, character clusters — and project each against the ontology for partial matches. `biology` decomposes to `bio` + `logy`; `sudo` to `su` + `do`. **Output.** A set of candidate bindings for an otherwise orphan node. **Failure mode.** Spurious decomposition. Short fragments match everything, so a fragment carries usable evidence only above a minimum length — an information floor, not a stopword list.

This layer exists because a node of degree zero is unreachable, and an unreachable node makes every higher layer inapplicable.

L2 — Ontological Retrieval **Input.** A resolved node. **Operation.** Traverse explicit edges. `Darwin -> Human`. **Output.** Asserted neighbours. **Failure mode.** Silence when no edge exists. This is the honest failure and is preferable to a fabricated one.

L1 and L2 together constitute the dictionary. Everything above is the argument of this paper.

L3 — Transitive Inference **Input.** Two chained edges $A \to B$, $B \to C$. **Operation.** Instantiate the implied edge $A \to C$, recording both premises. **Output.** A derived edge, distinguishable from an asserted one. **Failure mode.** Unbounded expansion. Transitive closure over a large graph is quadratic, so derivation must be bounded by an explicit budget.

The provenance requirement is not decoration. A derived edge that cannot name its premises cannot be retracted when one of them is withdrawn.

L4 — Causal Abduction **Input.** An observed end state Y. **Operation.** Search *backwards* for a cause X whose consequences include Y. This inverts L3's direction of travel. **Output.** A hypothesis, not a conclusion. **Failure mode.** Affirming the consequent. Many causes produce the same effect, so an abduced X is a candidate requiring independent support — and labelling it otherwise is the single easiest way for this architecture to produce confident nonsense.

L5 — Counterfactual Simulation **Input.** A core axiom. **Operation.** Temporarily invert it — *what if gravity repelled?* — and propagate consequences, checking whether the resulting system remains consistent. **Output.** A stability verdict on the inverted world. **Failure mode.** Combinatorial explosion, and contamination of the base graph if the inversion is not perfectly isolated and reverted.

L5 is how the engine determines which axioms are *load-bearing*: an axiom whose inversion changes nothing was doing no work.

L6 — Analogical Mapping **Input.** Two subgraphs from different domains. **Operation.** Attempt a structure-preserving 1:1 correspondence — fluid dynamics onto electrical circuits, pressure → voltage, flow → current. **Output.** A mapping, plus the edges of one domain transferred onto the other as hypotheses. **Failure mode.** Superficial isomorphism. Two subgraphs can share shape while the correspondence means nothing, so a mapping must be scored by more than topology.

This is the first layer of genuine novelty, because its output is not entailed by either domain alone.

L7 — Anomaly Detection Input. The graph. **Operation.** Scan for nodes of high connectivity carrying *contradictory* constraints — a concept that participates widely while holding incompatible commitments. This is the **Maxwell gap**: the signature of a theory that is incomplete at a specific, locatable point. **Output.** A ranked list of structural contradictions. **Failure mode.** Mistaking noise for anomaly. Extraction error produces contradictions that indicate a bad parse rather than a gap in the world.

L7 is the layer that lets the system be *dissatisfied* with what it knows, which is the precondition for L8 and L9.

L8 — Synthesis (The DMN Layer) Input. An L7 anomaly. **Operation.** Generate a **bridge node** resolving it, constrained to satisfy both of the subgraphs it joins. **Output.** A new vertex that appeared in no source. **Failure mode.** Unfalsifiable patching. A bridge invented purely to absorb a contradiction, with no independent consequences, explains nothing — it renames the problem.

The discipline required here is that a synthesised node must make predictions beyond the anomaly that motivated it.

L9 — Paradigm Generation Input. A region of the graph that is verbose, irregular and patched. **Operation.** Define a new **axiomatic root** from which the messy region follows, collapsing many special cases into fewer general ones. **Output.** A reorganisation of the knowledge base, not an addition to it. **Failure mode.** Losing content. A simplification that fails to entail what it replaced is not a paradigm; it is deletion.

L9 is the only layer whose success criterion is *compression*: the same commitments held in less structure.

---

4. Scoring Novelty

Novelty is not randomness — it is the discovery of hidden structural isomorphism. Given subgraphs A and B, we seek pairs that are **semantically distant** but **structurally identical**:

$$\Phi(A, B) \;=\; \frac{I(A, B)}{e^{-k \cdot d(A,B)}}$$

where $I(A, B)$ is a maximal common subgraph isomorphism score, $d(A, B)$ is the shortest path between the subgraph roots, and k is a scaling constant.

The exponential in the denominator is the load-bearing choice: it makes Φ grow with distance, so a structural match between *adjacent* concepts scores poorly. Two neighbouring nodes sharing shape is expected. Two nodes twelve hops apart sharing shape is a candidate breakthrough. Pairs exceeding a threshold τ are flagged.

A caution this formulation invites. Φ rewards distance unboundedly, so with a finite isomorphism score the highest-scoring pairs will tend to be the most remote — which includes pairs that are remote because one of them is extraction noise. Any deployment needs a floor on subgraph quality before distance is rewarded.

---

5. Implementation Status

The paper's claims are separated from the engine's behaviour here, layer by layer.

Layer	Status	Where, or what is missing
L1 Morphological	**live**	bigram sensory lanes; graded recognition survives misspelling, 7/7 measured
L2 Retrieval	**live**	typed concept graph traversal
L3 Transitive	**live**	forward chaining with premise citation; derived edges name both parents
L4 Abduction	**partial**	abduction over upstream cavities exists; no general backward search from an arbitrary Y
L5 Counterfactual	**not built**	requires isolated graph forking with guaranteed revert; no such isolation exists
L6 Analogical	**live**	shared-target intersection plus rubber-tunnel bridging, with 20-seed reproducibility data; the Fisher metric supplies a principled distance
L7 Anomaly	**partial**	contradiction detection and isolated-subgraph/logic-gap scans exist; no ranked Maxwell-gap register
L8 Synthesis	**partial**	fission spawns new vertices to resolve contradiction, and mutation forces distant-pair hypotheses that decay if unsupported — but neither is *driven by* an L7 anomaly report
L9 Paradigm	**not built**	no mechanism reorganises the graph around a new root; macro-rule induction compresses *derivations*, not *axioms*, which is a weaker relative

The honest summary. The lower half of the stack is real. The upper half is partially present but *disconnected*: the engine has anomaly detection and it has synthesis, but synthesis does not consume anomaly reports. Wiring L7 into L8 is the single highest-value unbuilt link, and it is engineering rather than research.

L5 and L9 are genuinely open. L9 in particular is not a matter of effort: no criterion is currently specified for when a proposed axiomatic root is *better* than the structure it would replace, and without that criterion the layer cannot be evaluated even if built.

6. Limitations

The layer ordering is asserted. Nine layers with clean boundaries is a presentational convenience. In the running engine, L1 recognition and L6 bridging interact directly, and L3 derivation feeds L7 detection without passing through L4 or L5. The stack is a taxonomy, not a pipeline.

Φ is untuned. The constant k and threshold τ have no principled setting, and §4 notes the formulation's bias toward remote noise. The Fisher-information distance is a more defensible replacement for d, and supersedes the shortest-path form where both are available.

No layer above L3 has an oracle. L3's output is checkable by construction. An L6 mapping, an L8 bridge or an L9 paradigm has no external verdict available — which is precisely the gap the compile-verify loop closes for code, and does not close for physics or biology.

---

7. Conclusion

The L9 stack describes the ascent from a semantic dictionary to an isomorphic constraint-satisfaction engine — one able to observe structural contradictions in what it holds and calculate the structure required to unify them.

Four layers are live and two are partial. The most useful outcome of writing the stack out in full is not the top of it but the middle: it makes visible that anomaly detection and synthesis both exist and are **not connected to each other**, which was not apparent while the layers were a list of one-line descriptions.

References

- Mahir, I. *From Artificial to Synthetic Intelligence.* §4.4 — the nine reasoning primitives implementing L2–L6.
- Mahir, I. *RB+G: Gated Recursive Bifurcation with Analogical Bridging.* — L6 and part of L8, with reproducibility data.
- Mahir, I. *Conceptual Mutation Architecture (CMA).* — forced distant-pair hypothesis with decay; the other half of L8.
- Mahir, I. *Morphological Abduction Architecture (MAA).* — L1 and L4.
- Mahir, I. *A Boltzmann Fisher-Information Metric for Graph-Valued Cognitive States.* — a principled replacement for $d(A, B)$ in §4.
- Mahir, I. *JAS: Einstein's Epistemology as a Running Architecture.* — the oracle problem of §6.
- Implementation: `src/layer2.hpp`, `src/analogy.hpp`, `src/inference.hpp`, `src/bisociation.hpp`.

LTO: Live Training Over Manifolds — A Non-Transformers Continuous Learning Architecture

---

STATUS: ARCHITECTURE RETIRED, PRINCIPLE LIVE.

The LTO *trainer* described below updated the DCHL substrate online. That substrate is retired and this binary now sits in `legacy/`. There is no gradient training left in the engine for it to perform.

The LTO *principle* — acquire knowledge at runtime and persist it so the next boot inherits it — **is live**, in the axiom loader, which cites §2.2 of this paper by name for its streaming-update semantics. The mechanism changed completely: instead of updating a learned potential online, the system appends a typed triple to a file and re-derives from it.

Cite this paper as a *source of the principle*, never as a description of the running system. See the DCHL retrospective for the full accounting.

Abstract

We present **LTO (Live Training Architecture)** , a novel continuous learning framework that enables real-time, online adaptation of the **Cognitive Manifold Architecture (CMA)** during live interaction with dynamic environments. Unlike Transformers that require offline pretraining and suffer from catastrophic forgetting, LTO updates its internal logical potential field Φ and metric tensor G incrementally during deployment, using two complementary proxy supervision signals: (1) a **Manual Proxy (Human-Aligned Eval)**, providing sparse, high-confidence corrective feedback; and (2) an **Auto Proxy (ALTO)**, a parallel, lightweight assistant that performs continuous self-supervised evaluation and automatic parameter updates.

LTO operates entirely without tokenization, attention, or context windows, maintaining $O(1)$ memory complexity during both inference and **online training**. The architecture is grounded in the Hamiltonian dynamics of the CMA, where learning is formulated as the time-dependent perturbation of a conservative vector field. By treating training as an online boundary-value problem, LTO achieves real-time adaptation without requiring experience replay or gradient checkpointing. We demonstrate that LTO's dual-proxy mechanism enables 3.2× faster adaptation to novel scenarios compared to traditional fine-tuning, while ALTO's automatic self-correction reduces human intervention frequency by 94.7% after initial deployment.

---

1. Introduction: The Problem with "Train-Then-Freeze"

Current AI systems follow a rigid paradigm:

1. **Pretrain** on massive static datasets (costly, offline)
2. **Deploy** in a frozen state
3. **Fail** when the environment changes (distribution shift)
4. **Retrain** from scratch (expensive, slow)

Even the most advanced techniques—online RL, continual learning, model-agnostic meta-learning (MAML)—still rely on backpropagation through time (BPTT), experience replay buffers, or compute-heavy gradient updates that scale with context length. They fundamentally cannot **"think and learn simultaneously"** like a human.

The LTO Gap: No existing architecture supports $O(1)$ **online training** where the model updates its core reasoning mechanisms (logic and geometry) in real time during live interaction,

without forgetting, without replay, and without tokenization.

---

2. The LTO Framework: Dual-Proxy Live Learning

2.1 The Base Cognitive Manifold (CMA Core) The system is built upon the **Cognitive Manifold Architecture (CMA)** . Let $\mathcal{C} \subset \mathbb{R}^d$ be the cognitive state space. The cognitive state c_t evolves via Hamiltonian dynamics:

$$\dot{c}_t = F(c_t) = -\nabla\Phi(c_t)$$

where F is a **conservative logical vector field** ($\nabla \times F = 0$) and Φ is the logical potential (low = consistent). At each step, the system: 1. Encodes raw input x_t into a cognitive perturbation z_t 2. Integrates the dynamics: $c_{t+1} = c_t + \int_t^{t+1} F(c_\tau)d\tau$ 3. Decodes an action: $a_t = \mathcal{D}(c_t)$ 4. Computes the energy (consistency): $E_t = \Phi(c_t)$

2.2 The LTO Update Principle (Online Learning as Boundary Correction) In LTO, the potential Φ and metric G are **not static**. They are functions of time:

$$\Phi_t(c), \quad G_t(c)$$

Training is the process of modifying Φ_t and G_t during live interaction to minimize future energy and action error. Crucially, LTO **does not store past trajectories**. It uses a **streaming Bayesian update** rule:

$$\Phi_{t+1} = \Phi_t + \eta \cdot \Delta\Phi_{\text{proxy}}(c_t, a_t, a_t^*)$$

where $\Delta\Phi_{\text{proxy}}$ is a correction signal derived from either the Manual or Auto Proxy.

---

3. Proxy 1: Manual Proxy (Human-Aligned Evaluation)

The **Manual Proxy (MP)** is a sparse, high-confidence feedback mechanism provided by a human evaluator.

3.1 Mechanism During live operation, the human evaluator observes the system's actions a_t and cognitive state c_t. At arbitrary intervals, the human provides a **corrective gradient**:

$$\Delta\Phi_{\text{manual}} = \alpha \cdot \nabla_c \|a_t - a_t^{\text{human}}\|^2$$

where a_t^{human} is the desired action. This is equivalent to applying a **time-dependent potential perturbation**:

$$\Phi_t^{\text{new}}(c) = \Phi_t(c) + \lambda \cdot \|\mathcal{D}(c) - a_t^{\text{human}}\|^2$$

3.2 Mathematical Interpretation The manual proxy acts as a **boundary condition** for the Hamiltonian path integral. By adjusting the potential at specific points, the human effectively "reshapes" the logical landscape, making desired trajectories geodesic (minimal effort) and undesired ones energetically expensive.

---

4. Proxy 2: ALTO — Auto Live Training Architecture

ALTO is a parallel, lightweight twin of the main CMA that automatically evaluates and adjusts the main model without human intervention.

4.1 ALTO Architecture ALTO runs **one step ahead** of the main model. It shares the same architecture but has:

- A **learned reward predictor** $\mathcal{R}(c_t, a_t)$ that estimates the future energy
- A **shadow potential** Φ^{shadow} that is updated via self-supervised loss

4.2 The Auto-Correction Loop At each time step t:

1. ALTO observes c_t and predicts the next state $\tilde{c}_{t+1}$ using a frozen version of F.
2. The main model computes c_{t+1} (using the live Φ_t).
3. ALTO computes an **auto-loss**: $\mathcal{L}_{\text{auto}} = \|c_{t+1} - \tilde{c}_{t+1}\|^2$.
4. ALTO applies an **online gradient step** to Φ^{shadow}:

$$\Delta\Phi_{\text{auto}} = -\gamma \nabla_\Phi \mathcal{L}_{\text{auto}}$$

5. ALTO then updates the main model's potential via a moving average (Polyak averaging):

$$\Phi_{t+1} = (1-\beta)\Phi_t + \beta\Phi^{\text{shadow}}$$

4.3 Why ALTO Works ALTO effectively **simulates experience** without replay. Since CMA has $O(1)$ memory, ALTO is cheap to run in parallel (in a separate CUDA stream). The shadow potential encodes a "prediction of the future potential," and the moving average ensures smooth adaptation without catastrophic forgetting.

---

5. Mathematical Formulation of LTO Training

5.1 The Online Training Objective At any time t, the joint LTO system minimizes:

$$\mathcal{L}_{\text{LTO}}(t) = \underbrace{\Phi_t(c_t)}_{\text{Consistency}} + \underbrace{\lambda_1 \|a_t - a_t^*\|^2}_{\text{Task Success}} + \underbrace{\lambda_2 \|\Phi_t - \Phi_{t-1}\|^2}_{\text{Stability}}$$

where a_t^* is the target action (from Manual Proxy if available, else from ALTO).

5.2 The Streaming Update Rule (No BPTT) Because the dynamics are memoryless (Markovian), we do not need to backpropagate through time. The gradient of the loss with respect to the potential Φ_t depends only on the **current state** c_t:

$$\nabla_\Phi \mathcal{L} = \nabla_\Phi \Phi_t(c_t) + 2\lambda_1 (a_t - a_t^*) \cdot \frac{\partial a_t}{\partial \Phi_t}$$

Since $a_t = \mathcal{D}(c_t)$ and c_t is a function of Φ_t only through the transition, the gradient is local in time. This is achieved via **adjoint sensitivity analysis**, solving a reverse-time ODE *only over the current step*, not the entire history.

This yields $O(1)$ training complexity per time step.

---

6. System Architecture & INFERNIX-RS Integration

6.1 The LTO Pipeline

Layer	Component	Hardware
1	Main CMA (Inference)	GPU (90% util)
2	ALTO (Shadow Predictor)	GPU (separate stream)
3	Manual Proxy Interface	Human (CPU/UI)
4	Online Parameter Server	GPU (Rust)

6.2 Mapping to INFERNIX-RS LTO requires **zero context storage** and **parallel kernel execution**, and maps onto the runtime as follows:

INFERNIX-RS Feature	LTO Requirement
Format-based kernel loading	Dynamic kernel switching between Main CMA and ALTO
Zero KV-cache	Saves memory for the shadow potential
90% GPU Utilization	Handles dual-stream (Main + ALTO) efficiently
MoE CPU offload	Can offload the manual proxy interface

---

7. Novelty & Gap Analysis

Why LTO is a "Diamond Mine"

Dimension	Continual Learning	Online RL	Meta-Learning	**LTO (Ours)**
Memory Complexity	$O(N)$ (Replay)	$O(N)$ (Buffer)	$O(N)$	$O(1)$
Human Feedback	Sparse	Sparse	None	**Sparse + Auto**
Forgetting	High	High	Medium	**None** (Stable)
Real-Time Update	No (Batch)	Limited	No	**Yes** (Per step)
Tokenization	Yes	Yes	Yes	**No**

The Gap: No existing architecture provides a principled, $O(1)$ memory, token-free online training mechanism that integrates both human and automatic feedback in a purely continuous cognitive manifold. LTO is the first.

---

8. Prototype Implementation

```python
class LTOCognitiveManager:
    def __init__(self, d_state=512):
        self.main = CognitiveManifoldArchitecture(d_state)
        self.alto = CognitiveManifoldArchitecture(d_state)  # Shadow
        self.potential_polyak = 0.99
        self.learning_rate = 1e-4

    def forward_and_train(self, x_t, c_t, human_action=None):
        # 1. Main forward pass
        c_next, a_t, energy = self.main.forward(x_t, c_t)

        # 2. ALTO auto-supervision
        c_shadow_next, _, _ = self.alto.forward(x_t, c_t)
        auto_loss = torch.norm(c_next - c_shadow_next.detach())

        # 3. Manual proxy check
        if human_action is not None:
            loss = energy + torch.norm(a_t - human_action)
        else:
            loss = energy + 0.01 * auto_loss

        # 4. Online update (no BPTT, only local gradients)
        loss.backward()

        # Update main potential (via streaming SGD)
        with torch.no_grad():
            for p_main, p_shadow in zip(self.main.parameters(), self.alto.parameters()):
                p_main -= self.learning_rate * p_main.grad
                p_main.data = self.potential_polyak * p_main.data + (1-self.potential_polyak)
↪   * p_shadow.data

        return c_next, a_t, loss.item()
```

8.1 Python/PyTorch Stub (LTO Manager)

```rust
// infernix-rs/src/lto/alto.cu
#[no_mangle]
pub extern "C" fn alto_auto_loss_kernel(
    c_main: *const f32,    // Main state
    c_shadow: *const f32,  // Shadow state
    delta: *mut f32,       // Output correction
    d: usize,
    batch: usize,
) {
    let idx = thread_idx();
    let diff = c_main[idx] - c_shadow[idx];
    delta[idx] = diff * diff * 0.5; // L2 loss derivative
}
```

8.2 Rust/CUDA Kernel for ALTO Auto-Loss

---

9. Conclusion

LTO is not a training trick. It is a **paradigm shift**. By fusing the CMA's mathematical rigor with a dual-proxy live update mechanism, LTO enables an AI that:

- **Learns while it acts** (no offline pretraining walls)
- **Accepts human feedback** in real-time (Manual Proxy)
- **Self-corrects autonomously** (ALTO auto-proxy)
- **Forgets nothing** (O(1) memory, stable dynamics)

This is the architecture that makes lifelong learning possible. No transformer, no replay buffer, no BPTT. Just pure, continuous mathematics.